COMPUTER SCIENCE + COMPUTER ENGINEERING

06

Laboratory Workbook

Implementation procedures and evidence contracts

20 original labs · 20 contract exercises · Five integrating projects · Failure-path practice

PACKAGE 06 / EXPANDED VOLUME 05 / SEPTEMBER 28, 2026
Self-study: derive, implement, test and explain.

Laboratory Workbook

Keep prediction, execution and explanation separate

Every lesson has an original assignment and an expanded five-step procedure. The supplied contract exercise isolates one testable skill; it does not replace the larger assignment. Complete both. Preserve the starter's public function signature and docstring, run the selected starter tests, add at least one new boundary test and explain the invariant.

The supplied test bank contains 120 table-driven checks plus 3 mutation/history checks across the 20 exercises. Cooker/BVH integration has a separate 20-case suite. These are regression checks, not an exhaustive proof. False confidence from passing an incomplete test set is itself a failure mode to investigate.

Workflow and evidence minimum

Before execution, write your expected numerical/structural result. During implementation, record exact commands, revision and environment. Afterward, preserve raw output, a deliberately triggered failure, the repaired test and a transfer problem solved without copying the reference. Use templates/LAB_JOURNAL.md.

For GPU labs, record device/driver/browser/SDK and validation output. Mark NOT RUN for unavailable hardware; a CPU serial-model test is not a GPU synchronization result. Use the native API documentation specified by your selected route.

A lesson passes only after its listed criteria, the original assignment and the independent explanation are satisfied. The first failing result is valuable evidence; do not delete it from the journal after finding the fix.

Lab 21.1 — Images, color and coordinate conventions

Original assignment (retained)

Write a PPM image containing a gradient and coordinate markers. Pass: the dimensions, orientation and channel values are verified by tests, and a README states all coordinate and color assumptions.

Expanded procedure

1. Write the image/coordinate contract before coding; include channel order, row origin, sample centers and the output transfer function.
2. Create a 320-by-180 buffer. Add a horizontal linear-light ramp and four distinct corner markers. Export RGB8 P6 PPM.
3. Parse the header and raster with an independent test. Check byte count and markers, including a padded-stride source.
4. Run the linear-average versus encoded-average experiment. Preserve its numeric values rather than only a screenshot.
5. Add invalid dimension, out-of-bounds coordinate and nonfinite color tests. Document the chosen failure behavior.

Pass criteria

- Exact dimensions and marker positions; no padding leaks into exported raster.
- Round-trip encode/decode within 1e-6 before byte quantization for representative values.
- Negative dimensions and invalid coordinates fail explicitly rather than silently wrapping.

Coding contract

Implement image_bytes(width, height, channels, row_alignment) with positive-integer validation and padded row accounting.

Edit `exercises/starter/lesson_21_1.py`. The function docstring and the selected test cases define its exact input/output contract. Read `exercises/reference/lesson_21_1.py` only after your own attempt.

```
python tests/test_exercises.py --track starter --lesson 21.1
```

Diagnose before changing code

Upside-down output usually indicates an origin mismatch. Blue/red swaps indicate channel order. A too-dark gradient often indicates missing output encoding; a washed-out image can indicate double encoding. First inspect stored values, then inspect the viewer.

Transfer and evidence

Add one new boundary or failure case. Record the prediction, raw output, explanation and retest in templates/LAB_JOURNAL.md. For Module 22, include actual hardware evidence or mark NOT RUN.

Lab 21.2 — Projection and clipping before perspective division

Original assignment (retained)

Implement clipping against one plane, then the complete chosen clip volume. Pass: triangles crossing the near plane remain finite and correctly shaped, and tests cover points exactly on the boundary.

Expanded procedure

1. Implement signed clip-plane distances and test points inside, outside and exactly on each plane.
2. Implement one-plane polygon clipping with complete vertices, including UV and color attributes.
3. Apply all six planes, fan-triangulate the result and divide only after clipping.
4. Move one triangle repeatedly across the near plane. Log output counts and assert every retained vertex is finite.
5. Test all-outside polygons, reversed edge traversal, a boundary vertex, degenerate geometry and rejected singular w.

Pass criteria

- Retained vertices satisfy all six inequalities within the declared tolerance.
- No infinity, NaN or enormous viewport allocation appears during near-plane crossing.
- Interpolated intersection attributes use the same t as the clip-space position.

Coding contract

Implement clip_near(polygon) for homogeneous tuples (x,y,z,w), retaining exact-boundary points and creating crossing intersections.

Edit `exercises/starter/lesson_21_2.py`. The function docstring and the selected test cases define its exact input/output contract. Read `exercises/reference/lesson_21_2.py` only after your own attempt.

```
python tests/test_exercises.py --track starter --lesson 21.2
```

Diagnose before changing code

Exploding triangles suggest division happened too soon. Missing near-plane triangles suggest whole-triangle rejection replaced clipping. Seams suggest inconsistent intersection or fill rules. A left/right mirror suggests matrix or viewport conventions, not a need to reverse all normals.

Transfer and evidence

Add one new boundary or failure case. Record the prediction, raw output, explanation and retest in templates/LAB_JOURNAL.md. For Module 22, include actual hardware evidence or mark NOT RUN.

Lab 21.3 — Triangle coverage, depth and perspective-correct interpolation

Original assignment (retained)

Render two triangles sharing an edge and a receding checkerboard. Pass: no cracks under the documented fill rule, correct overlap ordering, and a visible comparison between affine and perspective-correct texture coordinates.

Expanded procedure

1. Implement edge functions and a bounded raster loop, then add the documented half-open shared-edge rule.
2. Add a coverage-count target and test the exact 64-sample rectangle. Reverse winding and triangle order.
3. Add depth with an explicit clear value and comparison. Test near/far ordering in both submission orders.
4. Implement both affine and perspective-correct attributes and render the receding checkerboard in both modes.
5. Run clipped and degenerate triangles through the full pipeline and save comparison images plus numerical coverage evidence.

Pass criteria

- No shared-edge double hits or holes for the quantized rectangle test.
- Opaque overlap is draw-order independent except documented exact-depth ties.
- The numerical interpolation fixture yields 11/19 and depth 0.54.

Coding contract

Implement perspective_attribute(weights, values, clip_w), validating finite nonzero positive w and a usable denominator.

Edit `exercises/starter/lesson_21_3.py`. The function docstring and the selected test cases define its exact input/output contract. Read `exercises/reference/lesson_21_3.py` only after your own attempt.

```
python tests/test_exercises.py --track starter --lesson 21.3
```

Diagnose before changing code

Bending checkerboards usually indicate affine UVs. Random holes often indicate winding or edge-equality mistakes. Correct colors with wrong overlap suggest depth-space confusion. Do not loosen a global epsilon to conceal all three.

Transfer and evidence

Add one new boundary or failure case. Record the prediction, raw output, explanation and retest in templates/LAB_JOURNAL.md. For Module 22, include actual hardware evidence or mark NOT RUN.

Lab 21.4 — Rays, intersections and acceleration structures

Original assignment (retained)

Implement sphere or triangle intersections and a small BVH. Pass: accelerated and brute-force results agree within a defined tolerance over randomized rays, including misses and grazing cases; report build and query costs separately.

Expanded procedure

1. Implement a bounded ray and sphere intersection; verify the two direction-scale fixtures.
2. Implement slab AABB intersection with explicit parallel-axis handling.
3. Build a median-split sphere BVH and keep brute-force traversal independent.
4. Compare at least 1,000 seeded rays across empty, separated and overlapping scenes. Log seed, tolerance, hit ID and t mismatches.
5. Report build time, query time, node visits and primitive tests separately. Render or visualize one test scene.

Pass criteria

- Sphere and box fixtures match expected t intervals.
- BVH and brute-force agree on hit/miss and nearest hit within the declared tolerance.
- Empty scenes, parallel directions and inside-origin rays are handled without undefined arithmetic.

Coding contract

Implement ray_box(origin, direction, box_min, box_max) with a [0,infinity) interval and parallel-axis handling.

Edit `exercises/starter/lesson_21_4.py`. The function docstring and the selected test cases define its exact input/output contract. Read `exercises/reference/lesson_21_4.py` only after your own attempt.

```
python tests/test_exercises.py --track starter --lesson 21.4
```

Diagnose before changing code

False misses often come from nonconservative bounds or an incorrect parallel-axis case. Wrong nearest hits often come from returning the first visited leaf instead of the nearest candidate. Acceleration is secondary until the oracle comparison passes.

Transfer and evidence

Add one new boundary or failure case. Record the prediction, raw output, explanation and retest in templates/LAB_JOURNAL.md. For Module 22, include actual hardware evidence or mark NOT RUN.

Lab 22.1 — GPU work submission, resources and pipeline state

Original assignment (retained)

Create a window or offscreen target and report supported capabilities. Pass: render a clear color, capture validation output and explain which objects belong to CPU recording versus GPU execution.

Expanded procedure

1. Select one backend and record the exact toolchain and hardware/implementation available. Keep the CPU reference usable.
2. Create an offscreen target or window/canvas, query required limits and enable available validation.
3. Clear one frame and capture output. Label every created object and save errors, even when the log is empty.
4. Draw a timeline separating record, submit, complete and present. Identify the completion evidence for every reusable resource.
5. Test a missing or unsupported capability and orderly shutdown. Mark unexecuted device tests pending, not passed.

Pass criteria

- A capability report names the actual selected route and enabled capabilities.
- Clear output and validation evidence come from a real run or are explicitly marked pending.
- The learner can identify at least one allocation whose lifetime extends beyond CPU submission.

Coding contract

Implement can_retire(last_submission, completed_submission) for a monotonic single-queue completion ledger with validated nonnegative integers.

Edit `exercises/starter/lesson_22_1.py`. The function docstring and the selected test cases define its exact input/output contract. Read `exercises/reference/lesson_22_1.py` only after your own attempt.

```
python tests/test_exercises.py --track starter --lesson 22.1
```

Diagnose before changing code

A successful device handle with no image does not prove that the render pass executed. Check target format, attachment usage, command submission and error scopes in order. Avoid disabling validation to hide a warning.

Transfer and evidence

Add one new boundary or failure case. Record the prediction, raw output, explanation and retest in templates/LAB_JOURNAL.md. For Module 22, include actual hardware evidence or mark NOT RUN.

Lab 22.2 — Buffers, shaders, layouts and resource binding

Original assignment (retained)

Upload a transform and vertex colors, then add a bounded compute operation. Pass: host/shader layouts are documented, a readback test checks results, and non-multiple work sizes remain correct.

Expanded procedure

1. Write a byte-offset table for one vertex record and one transform uniform; assert sizes or inspect the exact upload array.
2. Upload vertex colors and a translation matrix. Verify an origin point lands at the predicted position.
3. Implement a bounded compute operation with ceil-divided group counts and a shader bounds check.
4. Read back N=1,63,64,65 and 1000 results. Treat N=0 as an explicit no-work case.
5. Intentionally break a binding or offset in a scratch copy; capture how validation or numeric tests detect it, then restore the correct code.

Pass criteria

- Offsets, stride, matrix order and binding slots are documented.
- Compute readback matches an independent CPU result for non-multiple sizes.
- No out-of-bounds access or missing required shader-layout check is ignored.

Coding contract

Implement dispatch_groups(element_count, group_size) using integer ceiling division and explicit zero-work behavior.

Edit `exercises/starter/lesson_22_2.py`. The function docstring and the selected test cases define its exact input/output contract. Read `exercises/reference/lesson_22_2.py` only after your own attempt.

```
python tests/test_exercises.py --track starter --lesson 22.2
```

Diagnose before changing code

Exploded geometry often comes from stride or matrix interpretation. Correct first elements with corrupted later elements often indicate array stride. A failure only at N=65 often indicates a missing bounds guard.

Transfer and evidence

Add one new boundary or failure case. Record the prediction, raw output, explanation and retest in templates/LAB_JOURNAL.md. For Module 22, include actual hardware evidence or mark NOT RUN.

Lab 22.3 — Synchronization, hazards and frames in flight

Original assignment (retained)

Implement a compute-to-draw dependency and two reusable frame contexts. Pass: validation reports no relevant hazards and a stress test never resets a context still used by the GPU.

Expanded procedure

1. Draw a dependency graph for upload, compute, draw and presentation, naming access types and ranges.
2. Implement a correct compute-to-draw dependency in the chosen backend; distinguish implicit WebGPU handling from native barriers.
3. Create two frame contexts with explicit completion state. Refuse to reset an incomplete context.
4. Stress context reuse, resource replacement and exit under load. Save validation plus frame-identity evidence.
5. For Vulkan, review present-wait semaphore reuse separately from command-buffer and frame-fence reuse.

Pass criteria

- A CPU-side negative test rejects reuse while last_use exceeds completed.
- Hardware logs show no relevant hazard when that route is executed; otherwise the check remains pending.
- The lifetime proof distinguishes frame contexts, presentation images and all queue users.

Coding contract

Implement choose_context(last_use_serials, completed_serial) to return the first reusable context or None without modifying the ledger.

Edit `exercises/starter/lesson_22_3.py`. The function docstring and the selected test cases define its exact input/output contract. Read `exercises/reference/lesson_22_3.py` only after your own attempt.

```
python tests/test_exercises.py --track starter --lesson 22.3
```

Diagnose before changing code

A flicker that disappears after wait-idle is a useful clue, not a finished fix. Find the specific missing dependency or premature overwrite. A native validation warning should not be dismissed because the equivalent browser example runs.

Transfer and evidence

Add one new boundary or failure case. Record the prediction, raw output, explanation and retest in templates/LAB_JOURNAL.md. For Module 22, include actual hardware evidence or mark NOT RUN.

Lab 22.4 — Presentation, resizing and device capability policy

Original assignment (retained)

Stress resize, minimize, restore and orderly shutdown. Pass: no stale size-dependent resources, no division by zero, and a readable capability report explains enabled features and fallbacks.

Expanded procedure

1. Add explicit surface and shutdown states and a required/optional/disabled feature table.
2. Use pixel drawable extents. Test zero extent, rapid resizing and restoration; invalidate temporal history on incompatible changes.
3. Recreate all size-dependent resources at a controlled boundary and retire the old set safely.
4. Inject failure after each initialization stage and test cleanup. For native Vulkan, test the acquire-out-of-date early-return path.
5. Capture an actual compatibility row and preserve untested combinations as pending. Document device-loss behavior.

Pass criteria

- No division by zero, stale target mismatch or unsignaled-fence deadlock occurs in supported stress paths.
- The report distinguishes logical size, drawable size and resource extent.
- Shutdown and partial initialization do not leak owned resources under the tested model.

Coding contract

Implement drawable_extent(width, height, scale, limit) returning None for a zero drawable and a clamped positive extent otherwise.

Edit `exercises/starter/lesson_22_4.py`. The function docstring and the selected test cases define its exact input/output contract. Read `exercises/reference/lesson_22_4.py` only after your own attempt.

```
python tests/test_exercises.py --track starter --lesson 22.4
```

Diagnose before changing code

A stretched image suggests aspect ratio or logical/pixel size confusion. A hang after minimizing can indicate a fence reset with no subsequent submission. Persistent ghosts after resize suggest history was not invalidated.

Transfer and evidence

Add one new boundary or failure case. Record the prediction, raw output, explanation and retest in templates/LAB_JOURNAL.md. For Module 22, include actual hardware evidence or mark NOT RUN.

Lab 23.1 — Radiometry, reflectance and physically based materials

Original assignment (retained)

Implement diffuse lighting and one documented microfacet model. Pass: test a constant environment, grazing angles and roughness limits, and state approximations rather than labeling any shiny shader physically exact.

Expanded procedure

1. Write a units and space table for every lighting input. Implement Lambertian diffuse and verify the numerical irradiance fixture.
2. Implement one GGX/Smith/Schlick model with documented roughness encoding and dielectric/metallic policy.
3. Test normal transformations under nonuniform scale and reject invalid/zero-length direction vectors.
4. Build a material grid varying roughness and metallic input under a controlled light. Save raw linear outputs and display mapping settings.
5. Test grazing directions, no light and roughness limits. Explain the single-scattering approximation and the limits of the validation performed.

Pass criteria

- The diffuse fixture gives 1.59155 within the chosen tolerance.
- No NaN or infinity escapes at tested grazing or roughness limits.
- The report names the BRDF, parameter encoding, coordinate space and known approximation limits.

Coding contract

Implement lambert(rho, irradiance) with finite inputs, 0<=rho<=1 and nonnegative irradiance.

Edit `exercises/starter/lesson_23_1.py`. The function docstring and the selected test cases define its exact input/output contract. Read `exercises/reference/lesson_23_1.py` only after your own attempt.

```
python tests/test_exercises.py --track starter --lesson 23.1
```

Diagnose before changing code

Plastic-looking metals often indicate an unintended diffuse lobe. Highlights that move under object scaling suggest incorrect normal transforms. A roughness slider with an inverted effect may have confused roughness, smoothness and alpha.

Transfer and evidence

Add one new boundary or failure case. Record the prediction, raw output, explanation and retest in templates/LAB_JOURNAL.md. For Module 22, include actual hardware evidence or mark NOT RUN.

Lab 23.2 — Textures, mipmaps, filtering and tangent-space detail

Original assignment (retained)

Render a receding textured floor with and without mipmaps and inspect a normal-mapped seam. Pass: report texture storage including mip levels and explain why non-color data needs its own filtering policy.

Expanded procedure

1. Generate a mip chain for a small known color texture in linear light and verify exact dimensions and texel counts.
2. Render a receding checkerboard with nearest sampling, bilinear sampling and mip sampling at identical resolution.
3. Add a tangent-space normal test with a mirrored UV island and explicit handedness.
4. Record texture payload, selected levels and any supported anisotropic filtering setting. Include a short repeatable camera-motion sequence.
5. Test one non-color map and explain its filtering policy separately from the color map.

Pass criteria

- A 256-square RGBA8 chain has 87,381 texels and 349,524 payload bytes.
- The seam test correctly accounts for tangent handedness.
- Color mip generation and non-color data handling are distinct and reproducible.

Coding contract

Implement mip_texels(width, height) using floor-halving to a minimum of one on each axis until both reach one.

Edit `exercises/starter/lesson_23_2.py`. The function docstring and the selected test cases define its exact input/output contract. Read `exercises/reference/lesson_23_2.py` only after your own attempt.

```
python tests/test_exercises.py --track starter --lesson 23.2
```

Diagnose before changing code

A stable blur is not proof that aliasing was solved correctly. Inspect motion and level selection. A single mirrored seam usually points to tangent conventions; uniformly wrong normal directions can instead indicate channel or color decoding.

Transfer and evidence

Add one new boundary or failure case. Record the prediction, raw output, explanation and retest in templates/LAB_JOURNAL.md. For Module 22, include actual hardware evidence or mark NOT RUN.

Lab 23.3 — Shadows, light selection and rendering-path tradeoffs

Original assignment (retained)

Implement shadow mapping and compare a simple forward path with tiled light selection or a small deferred prototype. Pass: disclose buffer sizes, light counts, transparency limitations and measured frame costs.

Expanded procedure

1. Implement a single shadow map with an explicit light transform, depth convention and outside-coverage policy.
2. Test acne and detachment on a flat plane, varying bias while holding everything else fixed.
3. Implement a simple forward baseline and one light-selection or deferred comparison path.
4. Test zero, few and many lights, plus list overflow and one transparency limitation.
5. Record attachment payload, actual light candidates, CPU/GPU timing scopes and equal-quality comparison images.

Pass criteria

- The shadow test demonstrates both insufficient and excessive bias without hiding either.
- The comparison keeps scene and quality fixed and does not silently discard overflowing lights.
- Reported bandwidth arithmetic is distinguished from hardware measurement.

Coding contract

Implement attachment_bytes(width, height, bytes_per_pixel, samples) with positive integer validation.

Edit `exercises/starter/lesson_23_3.py`. The function docstring and the selected test cases define its exact input/output contract. Read `exercises/reference/lesson_23_3.py` only after your own attempt.

```
python tests/test_exercises.py --track starter --lesson 23.3
```

Diagnose before changing code

Floating shadows suggest excessive bias or a depth-sign error. Shadows only on one side suggest light projection or clipping. A faster clustered path with missing lights is a correctness failure until a deliberate quality policy is documented.

Transfer and evidence

Add one new boundary or failure case. Record the prediction, raw output, explanation and retest in templates/LAB_JOURNAL.md. For Module 22, include actual hardware evidence or mark NOT RUN.

Lab 23.4 — HDR, tone mapping, temporal reconstruction and validation

Original assignment (retained)

Add exposure, a documented tone mapper and a basic temporal accumulation experiment. Pass: capture a camera cut and disocclusion test, show failure modes, and compare both still-image error and motion behavior.

Expanded procedure

1. Add a floating-point linear intermediate, exposure in stops and the documented teaching tone mapper.
2. Keep raw HDR values and encoded previews separate. Test values 0,0.18,1,4 and a nonfinite input.
3. Implement a basic temporal accumulation experiment with a visible history-valid flag.
4. Run camera-cut, disocclusion, resize and fast-motion sequences; capture failure cases as well as corrected output.
5. Report a still-image error metric in a stated domain and describe motion behavior separately.

Pass criteria

- Exposure -1 stop maps input 4 to exposed 2, tone-mapped 2/3 and encoded about 0.836.
- Camera-cut fixture outputs 0.2 rather than the stale-history result 0.92.
- The report distinguishes internal HDR, SDR presentation, tone mapping and temporal reconstruction.

Coding contract

Implement temporal_sample(current, history, weight, history_valid) with finite values and a weight constrained to [0,1].

Edit `exercises/starter/lesson_23_4.py`. The function docstring and the selected test cases define its exact input/output contract. Read `exercises/reference/lesson_23_4.py` only after your own attempt.

```
python tests/test_exercises.py --track starter --lesson 23.4
```

Diagnose before changing code

A dark output can result from encoding being skipped; fading trails indicate stale correspondence. A metric improving while thin objects vanish suggests the chosen metric is not sufficient to judge the temporal failure.

Transfer and evidence

Add one new boundary or failure case. Record the prediction, raw output, explanation and retest in templates/LAB_JOURNAL.md. For Module 22, include actual hardware evidence or mark NOT RUN.

Lab 24.1 — Fixed updates, interpolation and overload policy

Original assignment (retained)

Implement an accumulator loop and inject long frames. Pass: simulation advances consistently under normal load, interpolation is bounded, and the overload behavior is tested rather than silently spiraling.

Expanded procedure

1. Implement a fixed-step accumulator with previous/current snapshots and a documented integer or floating time representation.
2. Assign discrete and held input consistently to ticks. Add a deterministic tick-indexed input fixture.
3. Implement and name an overload policy; track dropped time rather than hiding it.
4. Test the 25-ms and 105-ms fixtures and a constant-velocity world with different normal rendering cadences.
5. Report simulation state separately from interpolated presentation, and state the supported determinism scope.

Pass criteria

- 25 ms with h=10 produces two steps, remainder 5 and alpha 0.5.
- 105 ms with h=10 and cap four produces four steps, remainder 5 and dropped 60 under the teaching policy.
- The time-accounting identity holds and alpha remains in [0,1).

Coding contract

Implement step_budget(accumulator_us, elapsed_us, step_us, max_steps) returning steps, remainder_us and dropped_us under the documented whole-tick drop policy.

Edit `exercises/starter/lesson_24_1.py`. The function docstring and the selected test cases define its exact input/output contract. Read `exercises/reference/lesson_24_1.py` only after your own attempt.

```
python tests/test_exercises.py --track starter --lesson 24.1
```

Diagnose before changing code

Objects that speed up at high frame rates may still use callback delta inside fixed rules. A jump applied several times after a stall suggests discrete input was not consumed per tick. Smooth output can hide dropped simulation time; inspect counters.

Transfer and evidence

Add one new boundary or failure case. Record the prediction, raw output, explanation and retest in templates/LAB_JOURNAL.md. For Module 22, include actual hardware evidence or mark NOT RUN.

Lab 24.2 — Entities, components, handles and data-oriented storage

Original assignment (retained)

Implement a small generational entity store with two components and deferred destruction. Pass: stale handles fail safely, iteration survives queued changes, and benchmarks include creation, deletion and common queries.

Expanded procedure

1. Implement a generational entity store with checked lookup and a generation-exhaustion policy.
2. Add position and velocity components. Choose AoS or SoA based on a written expected query.
3. Queue structural changes during iteration and apply them at a defined boundary.
4. Test slot reuse, stale handle, duplicate destruction, invalid index and conflicting deferred commands.
5. Benchmark creation, destruction and a position/velocity query with a reproducible dataset. Keep correctness tests enabled separately.

Pass criteria

- Handle (7,3) is rejected after slot 7 advances to generation 4.
- Deferred mutation neither skips nor repeats unintended entities.
- Durable save identity is not an unchecked raw pointer or dense-array index.

Coding contract

Implement handle_valid(index, generation, slots), where each slot stores alive and generation, with range-safe lookup.

Edit `exercises/starter/lesson_24_2.py`. The function docstring and the selected test cases define its exact input/output contract. Read `exercises/reference/lesson_24_2.py` only after your own attempt.

```
python tests/test_exercises.py --track starter --lesson 24.2
```

Diagnose before changing code

A deletion bug that appears only when removing a middle element often indicates an uncorrected reverse mapping. A stale handle that resolves after reuse means the generation comparison or alive check is missing.

Transfer and evidence

Add one new boundary or failure case. Record the prediction, raw output, explanation and retest in templates/LAB_JOURNAL.md. For Module 22, include actual hardware evidence or mark NOT RUN.

Lab 24.3 — Allocators, task graphs and frame-resource lifetime

Original assignment (retained)

Create a serial-then-parallel task graph and a checked temporary arena. Pass: resetting live storage is detected in tests, dependency order is verified, and the critical path is identified from a trace.

Expanded procedure

1. Implement a checked aligned byte arena with nonmutating failure and a live-use reset guard.
2. Create a serial topological graph executor with unknown-dependency and cycle detection.
3. Declare task reads/writes and identify which independent tasks truly access disjoint or read-only data.
4. Add a small parallel executor in the full lab, compare results to the serial oracle and collect task traces.
5. Test resetting live storage, missing dependencies and a cycle. Compute the example critical-path bound before measuring.

Pass criteria

- The alignment fixture starts at 32 and ends at 40; failed allocations preserve the old offset.
- Cycles and missing prerequisite names produce readable failures.
- Arena reset waits for every declared consumer; parallel output matches the reference under the tested model.

Coding contract

Implement topo_order(dependencies) with deterministic ready-task order, unknown-node checks and cycle rejection.

Edit `exercises/starter/lesson_24_3.py`. The function docstring and the selected test cases define its exact input/output contract. Read `exercises/reference/lesson_24_3.py` only after your own attempt.

```
python tests/test_exercises.py --track starter --lesson 24.3
```

Diagnose before changing code

Random data corruption after a frame boundary often indicates premature arena reuse. Poor scaling with idle workers can indicate a long critical path, not a broken thread pool. Shared writes without dependency or partitioning are races regardless of the graph's acyclic shape.

Transfer and evidence

Add one new boundary or failure case. Record the prediction, raw output, explanation and retest in templates/LAB_JOURNAL.md. For Module 22, include actual hardware evidence or mark NOT RUN.

Lab 24.4 — Subsystem boundaries, save formats and reloadability

Original assignment (retained)

Add versioned saves and one reloadable asset type. Pass: old supported data migrates, malformed data is rejected without partial world corruption, and subsystem shutdown works after injected initialization failure.

Expanded procedure

1. Draw subsystem boundaries with ownership, thread affinity and failure behavior for each interface.
2. Define a versioned save schema with stable IDs, finite transforms, size limits and explicit unsupported-version behavior.
3. Implement one migration and a candidate-world load path that commits only after full validation.
4. Implement one asset reload with revision tokens, preserving the old asset when the candidate fails.
5. Inject malformed saves and partial initialization failures; verify old-world preservation and cleanup of exactly the resources that were created.

Pass criteria

- Supported legacy data migrates predictably and unsupported versions fail clearly.
- Malformed data does not partially mutate the live world.
- A stale reload completion cannot replace a newer revision.

Coding contract

Implement validate_scene(document) to migrate the declared version-1 fixture to version 2 and reject duplicate IDs, nonfinite positions and unsupported versions.

Edit `exercises/starter/lesson_24_4.py`. The function docstring and the selected test cases define its exact input/output contract. Read `exercises/reference/lesson_24_4.py` only after your own attempt.

```
python tests/test_exercises.py --track starter --lesson 24.4
```

Diagnose before changing code

A save that works only in the same process may contain pointers or runtime indices. A failed load that leaves missing entities is not transactional. A newly edited texture reverting later suggests out-of-order reload completion.

Transfer and evidence

Add one new boundary or failure case. Record the prediction, raw output, explanation and retest in templates/LAB_JOURNAL.md. For Module 22, include actual hardware evidence or mark NOT RUN.

Lab 25.1 — Importing, cooking and content-addressed asset builds

Original assignment (retained)

Build a cooker for one mesh and one image representation. Pass: identical declared inputs reproduce the same result, changed settings invalidate the cache, and corrupted sources produce readable failures.

Expanded procedure

1. Define and validate one mesh source and one image source, with units, color semantics and bounded sizes.
2. Implement a cooker key containing all semantic inputs, dependencies, settings and tool version.
3. Cook into temporary files, validate and publish complete artifacts with hashes in a manifest.
4. Compare two clean cooks, an unchanged warm run, a setting change and a dependency change.
5. Inject malformed source and corrupt cached output; preserve the previous valid artifact or rebuild safely with a readable failure report.

Pass criteria

- Identical declared inputs reproduce identical tested artifact bytes.
- Settings and dependency changes invalidate affected keys.
- Corrupt sources or artifacts cannot silently become accepted runtime data.

Coding contract

Implement asset_key(source_bytes, settings, dependencies, tool_version) with SHA-256 and a canonical structured descriptor.

Edit `exercises/starter/lesson_25_1.py`. The function docstring and the selected test cases define its exact input/output contract. Read `exercises/reference/lesson_25_1.py` only after your own attempt.

```
python tests/test_exercises.py --track starter --lesson 25.1
```

Diagnose before changing code

An asset that looks stale after editing usually indicates a missing dependency or setting in the key. A rebuild on every machine with unchanged semantic inputs may be hashing absolute paths or timestamps. A cache hit with corrupt bytes means existence was mistaken for integrity.

Transfer and evidence

Add one new boundary or failure case. Record the prediction, raw output, explanation and retest in templates/LAB_JOURNAL.md. For Module 22, include actual hardware evidence or mark NOT RUN.

Lab 25.2 — Streaming, residency and memory budgets

Original assignment (retained)

Implement a bounded streaming queue with placeholder assets and cancellation. Pass: a forced small budget produces graceful fallback, no use-after-free, and a report of peak rather than only steady-state memory.

Expanded procedure

1. Define load states and which owner holds compressed, decoded, staging, resident and retiring storage in each.
2. Implement a bounded request queue with a placeholder and a request revision token.
3. Reserve peak memory before admission and distinguish combined teaching estimates from real CPU/GPU pool measurements.
4. Cancel at every stage, including after upload submission. Verify late completions never publish stale assets.
5. Force a budget too small for one request; verify bounded retry behavior, graceful fallback and a peak-memory report.

Pass criteria

- The 110-MiB peak request is rejected under a 100-MiB combined budget.
- Cancelled in-flight work retains storage until its consumers finish.
- Late completions cannot overwrite a newer request or remove its ownership state.

Coding contract

Implement admit_bytes(budget, resident, existing_temporary, new_staging, new_resident) using validated nonnegative integer byte counts.

Edit `exercises/starter/lesson_25_2.py`. The function docstring and the selected test cases define its exact input/output contract. Read `exercises/reference/lesson_25_2.py` only after your own attempt.

```
python tests/test_exercises.py --track starter --lesson 25.2
```

Diagnose before changing code

An out-of-memory event despite low resident usage often comes from overlapping decode/staging/retirement allocations. Assets reappearing after scene changes suggest missing request-generation checks. A cancellation crash often indicates storage was freed before a consumer stopped.

Transfer and evidence

Add one new boundary or failure case. Record the prediction, raw output, explanation and retest in templates/LAB_JOURNAL.md. For Module 22, include actual hardware evidence or mark NOT RUN.

Lab 25.3 — Editors, inspection and undoable commands

Original assignment (retained)

Create a scene inspector with transform editing and a ten-command undo/redo test. Pass: invalid inputs are rejected, focus does not trigger unintended gameplay, and save/reload preserves committed changes.

Expanded procedure

1. Implement a scene inspector that shows stable entity IDs, committed transforms and resource status.
2. Route edits through a validated before/after command, separating drag preview from commit and cancel.
3. Implement undo and redo with new-edit branch invalidation and an explicit scene-load history policy.
4. Run ten edits, ten undos and ten redos; then test undo followed by a new branch and invalid input.
5. Exercise field focus, camera controls and save/reload. Preserve only committed authoring changes.

Pass criteria

- Ten-command undo/redo round-trips return the expected logical states.
- A new edit after undo clears the ordinary redo branch.
- Invalid input leaves model and history unchanged; focused editing does not trigger gameplay.

Coding contract

Implement apply_edit(state, undo_stack, redo_stack, entity_id, new_position) so invalid input is nonmutating and a valid new edit clears redo.

Edit `exercises/starter/lesson_25_3.py`. The function docstring and the selected test cases define its exact input/output contract. Read `exercises/reference/lesson_25_3.py` only after your own attempt.

```
python tests/test_exercises.py --track starter --lesson 25.3
```

Diagnose before changing code

Undo that works once but not after scene load may use stale runtime handles. A camera moving while typing indicates missing focus routing. An enormous undo stack after one drag suggests previews were committed individually.

Transfer and evidence

Add one new boundary or failure case. Record the prediction, raw output, explanation and retest in templates/LAB_JOURNAL.md. For Module 22, include actual hardware evidence or mark NOT RUN.

Lab 25.4 — Engine integration and reproducible demonstration

Original assignment (retained)

Complete Gate E using the project specification in the handbook. Pass: another person follows the README, a headless test scene passes, all assets have provenance, and a limitations section identifies remaining correctness or performance risks.

Expanded procedure

1. Assemble the five module projects into the Gate E engine; retain the CPU reference and declare the selected GPU route or pending hardware work.
2. Write clean build/cook/run/test commands and a compatibility/feature matrix. Include asset provenance and exact quality settings.
3. Run the adversarial integration matrix, preserve raw timing samples and measure peak memory using an identified method.
4. Have a second person perform the clean build and reproduce a boundary test, edit and save/reload. Record their actual result.
5. Complete the oral review and limitations document; do not claim unrun hardware or reviewer checks as passed.

Pass criteria

- The README enables a real clean-source build and a documented second-person run.
- All mandatory correctness cases pass or unsupported GPU work is clearly excluded under the software-only allowance.
- Frame/memory budgets, raw evidence, provenance and limitations accompany the engine.

Coding contract

Implement nearest_rank(samples, percentile) with finite nonempty samples and 0<percentile<=100; document that this is not interpolated quantiles.

Edit `exercises/starter/lesson_25_4.py`. The function docstring and the selected test cases define its exact input/output contract. Read `exercises/reference/lesson_25_4.py` only after your own attempt.

```
python tests/test_exercises.py --track starter --lesson 25.4
```

Diagnose before changing code

A result that only works in the original working directory usually depends on implicit paths. A smooth average with obvious stutter requires raw frame distributions. A successful build without assets is not a reproducible engine demonstration.

Transfer and evidence

Add one new boundary or failure case. Record the prediction, raw output, explanation and retest in templates/LAB_JOURNAL.md. For Module 22, include actual hardware evidence or mark NOT RUN.

Module project 21 — CPU reference renderer

Budget allocation: 90-140 planning hours. The module project integrates its four labs.

Original brief: Build a CPU renderer that writes a portable image file. It must clip geometry, rasterize triangles, resolve visibility and interpolate attributes correctly. Keep it as a reference implementation for future GPU work.

Milestones

1. P21-A: image contract and byte-checked PPM output.
2. P21-B: six-plane homogeneous clipping with attributed vertices.
3. P21-C: quantized coverage, depth and perspective-correct UVs with comparison images.
4. P21-D: a sphere or triangle ray path and BVH/brute-force comparison.

Evidence package

- Coordinate contract; unit tests; clipped polygon dumps; coverage-count proof.
- Affine/corrected images; near-plane sequence; BVH seed and raw build/query measurements.

Review protocol

Demonstrate the smallest valid happy path, a boundary path and a deliberately injected failure. Explain ownership and arithmetic before showing timing. Answer the five module-review questions from the handbook, then compare with the separate key. Preserve failed and corrected evidence.

Mark each milestone NOT STARTED, IN PROGRESS, PASS or BLOCKED; blocked GPU work must retain a hardware-access note. Carry these records into Gate E rather than constructing five disconnected projects.

Module project 22 — One validated GPU route

Budget allocation: 90-140 planning hours. The module project integrates its four labs.

Original brief: Implement a GPU backend for the reference scene. Query device capabilities, enable available validation, document the toolchain and handle resize and shutdown safely. A CPU backend remains a valid fallback while hardware access is arranged.

Milestones

1. P22-A: capability report, clear target and validation capture.
2. P22-B: geometry, transform layout and bounded compute readback.
3. P22-C: compute-to-draw dependency and two reusable contexts.
4. P22-D: resizing, loss policy and failure-safe teardown.

Evidence package

- Actual backend/toolchain record; byte layout table; shader sources; readback results.
- Dependency ledger; frame-context stress log; resize/minimize matrix; pending hardware markers where applicable.

Review protocol

Demonstrate the smallest valid happy path, a boundary path and a deliberately injected failure. Explain ownership and arithmetic before showing timing. Answer the five module-review questions from the handbook, then compare with the separate key. Preserve failed and corrected evidence.

Mark each milestone NOT STARTED, IN PROGRESS, PASS or BLOCKED; blocked GPU work must retain a hardware-access note. Carry these records into Gate E rather than constructing five disconnected projects.

Module project 23 — Material and lighting laboratory

Budget allocation: 90-140 planning hours. The module project integrates its four labs.

Original brief: Create a material-and-lighting demonstration with a documented color pipeline. Compare image quality and frame cost at identical resolution and scene content; include reference images, not only attractive screenshots.

Milestones

1. P23-A: linear-light diffuse and documented microfacet material.
2. P23-B: semantic mip filtering and tangent-handedness tests.
3. P23-C: one shadow map and an equal-quality lighting-path comparison.
4. P23-D: exposure, tone mapping and temporal failure/rejection sequences.

Evidence package

- Units/space table; raw linear fixtures; material grid; normal seam test.
- Buffer budget; light-count data; shadow-bias comparison; temporal sequence and error domain.

Review protocol

Demonstrate the smallest valid happy path, a boundary path and a deliberately injected failure. Explain ownership and arithmetic before showing timing. Answer the five module-review questions from the handbook, then compare with the separate key. Preserve failed and corrected evidence.

Mark each milestone NOT STARTED, IN PROGRESS, PASS or BLOCKED; blocked GPU work must retain a hardware-access note. Carry these records into Gate E rather than constructing five disconnected projects.

Module project 24 — Headless engine core

Budget allocation: 80-120 planning hours. The module project integrates its four labs.

Original brief: Build a small engine with a fixed-step world, generational entity handles, explicit resource ownership and an inspectable task graph. Keep the game rules separate enough to run automated headless tests.

Milestones

1. P24-A: fixed ticks, interpolation and explicit overload accounting.
2. P24-B: generational identity and deferred structural edits.
3. P24-C: checked arena plus serial/parallel task-graph validation.
4. P24-D: schema migration, transactional loads and safe reload/shutdown.

Evidence package

- Tick-indexed input log; time invariant tests; entity mutation tests.
- Task trace and critical path; arena guard tests; save fixtures; stale reload and initialization-failure logs.

Review protocol

Demonstrate the smallest valid happy path, a boundary path and a deliberately injected failure. Explain ownership and arithmetic before showing timing. Answer the five module-review questions from the handbook, then compare with the separate key. Preserve failed and corrected evidence.

Mark each milestone NOT STARTED, IN PROGRESS, PASS or BLOCKED; blocked GPU work must retain a hardware-access note. Carry these records into Gate E rather than constructing five disconnected projects.

Module project 25 — Gate E integrated engine

Budget allocation: 70-110 planning hours. The module project integrates its four labs.

Original brief: Gate E: deliver a small 3D engine demonstration with an asset cooker, reproducible scene, inspector, save/load, diagnostics and a measured frame/memory budget. A second person must build and run it from the README.

Milestones

1. P25-A: reproducible mesh/image cooking with dependency-aware keys.
2. P25-B: bounded loading, cancellation and peak-budget evidence.
3. P25-C: validated inspector commands and undo/redo histories.
4. P25-D: clean build, measurements, second-person review and Gate E defense.

Evidence package

- Source/cooked manifest and provenance; cold/warm/change/corruption tests.
- Peak residency ledger; ten-command history tests; raw timings; reviewer record; explicit limitations.

Review protocol

Demonstrate the smallest valid happy path, a boundary path and a deliberately injected failure. Explain ownership and arithmetic before showing timing. Answer the five module-review questions from the handbook, then compare with the separate key. Preserve failed and corrected evidence.

Mark each milestone NOT STARTED, IN PROGRESS, PASS or BLOCKED; blocked GPU work must retain a hardware-access note. Carry these records into Gate E rather than constructing five disconnected projects.